

第三讲 - 列表 (编程练习 1-2-3)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



- 1 序列概述
- 2 创建、索引与切片
- 3 修改、增加与删除
- 4 查询、统计与排序
- 5 引用赋值与列表复制
- 6 嵌套列表与综合实践
- 7 总结与课后实践
- 8 课后拓展

为什么需要列表

某跨境电商团队正在维护计划进入的目标市场。

使用多个变量分别记录

```
market_1 = "泰国"  
market_2 = "越南"  
market_3 = "马来西亚"  
market_4 = "新加坡"
```

- 每增加一个市场，就要创建一个新变量；
- 这些变量无法作为一个整体进行操作；
- 市场数量较多时，代码难以维护。

使用一个列表统一记录

```
markets = [  
    "泰国",  
    "越南",  
    "马来西亚",  
    "新加坡"  
]
```

- 一个变量保存整份市场清单；
- 清单中的市场保持一定顺序；
- 后续可以统一查询和修改。

列表解决的问题

当多项数据具有相同的业务含义，并且需要作为一个整体进行处理时，可以使用列表保存这些数据。

序列与列表

序列是一组按照一定顺序排列的数据。序列中的每个元素都有自己的位置，Python可以通过整数索引定位其中的元素。常见的序列数据类型如下：

类型	示例	适合表示	能否修改
字符串	"Python"	一段文本	否
列表	["泰国", "越南"]	一组可能变化的数据	是
元组	("中国", "杭州")	内容固定的记录	否

除列表外，字符串和元组也是序列，但它们的内容不能直接修改，

列表的三个核心特征

- 有序：每个元素都有确定的位置，可以通过索引访问；
- 可变：创建列表后，仍然可以增加、删除或修改元素；
- 可保存多个元素：语法上允许保存不同数据类型的元素。

本讲示例任务：维护目标市场清单

本讲将围绕变量 `markets`，完成一份目标市场清单的维护任务。

任务	需要解决的问题	主要工具
保存数据	如何用一个变量保存多个目标市场	方括号 <code>[]</code> 、 <code>list()</code>
定位数据	如何取得某个市场或一些市场	索引、切片
更新数据	如何修改、增加或删除市场	索引赋值、列表方法、 <code>del</code>
查询与排序	如何判断市场是否存在，并调整排列顺序	<code>in</code> 、 <code>len()</code> 、 <code>sort()</code> 、 <code>sorted()</code>
保护原数据	如何建立新版本，同时保留原来的清单	赋值、列表复制、浅复制

贯穿本讲的两个思考要点

学习每一种列表操作时，都要先判断：**原列表是否改变**，以及**列表操作语句执行后会得到什么结果**。运行代码后，再核对自己的判断。

创建列表

Python使用方括号 `[]` 表示列表，各个元素之间使用英文逗号分隔。

```
# 目标市场清单
markets = ["泰国", "越南", "马来西亚", "新加坡"]
# 每日销售额清单
daily_sales = [12800, 9600, 13500, 8700]
# 创建空列表
pending_markets = []
another_empty_list = list()
```

列表的基本语法

- 列表以方括号开始和结束；
- 元素之间使用逗号分隔；
- 元素按照写入顺序排列；
- 空列表中暂时没有元素。

变量名表达业务含义

- `markets` 表示多个市场；
- `daily_sales` 表示多天销售额；
- 复数形式通常表示一组数据；
- 名称应说明列表保存的内容。

通过索引定位一个元素

列表中的每个元素都有两个可用的位置编号。

元素	泰国	越南	马来西亚	新加坡
正索引	0	1	2	3
负索引	-4	-3	-2	-1

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]
```

```
print(markets[0])    # 泰国  
print(markets[2])    # 马来西亚  
print(markets[-1])   # 新加坡  
print(markets[-2])   # 马来西亚
```

- 正索引从左向右计数；
- 第一个元素的索引是 0；
- 负索引从右向左计数；
- 最后一个元素的索引是 -1。

索引的有效范围

这份列表包含4个元素，因此正索引为 0 至 3，负索引为 -4 至 -1。超出范围会导致程序报错。

基本切片：取得一段连续数据

索引用于取得一个元素，切片用于从列表中取得一段元素。

列表名[start:stop]

```
markets = ["泰国", "越南", "马来西亚", "新加坡", "菲律宾", "印度尼西亚"]  
  
priority_markets = markets[1:4]  
  
print(priority_markets)  
# 输出: ['越南', '马来西亚', '新加坡']
```

start指定起点

markets[1:4] 从索引 1 开始，因此结果中包含“越南”。

stop指定终点

切片在索引 4 之前停止，因此结果中不包含索引 4 的“菲律宾”。

切片包含 start 位置，不包含 stop 位置。切片会产生一个新列表，原来的列表不变。

切片的省略、步长与逆序

```
markets = ["泰国", "越南", "马来西亚", "新加坡", "菲律宾", "印度尼西亚"]
```

```
print(markets[:3])    # 从开头取到索引3之前
print(markets[3:])    # 从索引3取到列表末尾
print(markets[::2])   # 从开头每隔一个取一个
print(markets[::-1])  # 按相反顺序生成新列表
```

切片写法	含义
<code>markets[:3]</code>	省略起点，默认从列表开头开始
<code>markets[3:]</code>	省略终点，默认一直取到列表末尾
<code>markets[::2]</code>	步长为2，每隔一个元素取一次
<code>markets[::-1]</code>	步长为负数，按照相反方向取值

切片的阅读顺序

先确定从哪里开始，再确定在哪里停止，最后判断每次移动的步长（默认为1）。省略的起点和终点由Python根据步长方向自动确定。思考 `markets[-3:]` 的结果。

修改已有元素

修改一个元素

已知元素的位置时，通过索引 **重新赋值**。

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]  
  
markets[2] = "马来西亚"  
print(markets)
```

修改连续多个元素

通过切片赋值，替换列表中的元素片段。

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]  
  
markets[1:3] = ["菲律宾",  
    ↪ "印度尼西亚"]  
print(markets)
```

观察重点

索引赋值和切片赋值都会 **直接修改原列表**；切片两侧的元素数量不必相同，故 **原列表长度也可能发生变化**。思考 `markets[1:3] = ["菲律宾"]` 和 `markets[3:3] = ["菲律宾"]` 的结果。

append()与extend(): 增加一个还是多个

增加一个市场

append 把一个对象作为新的列表元素。

```
markets = ["泰国", "越南"]  
  
markets.append("新加坡")  
  
print(markets)
```

增加多个市场

extend 依次加入另一个序列中的元素。

```
markets = ["泰国", "越南"]  
  
markets.extend(["新加坡",  
               ↪ "菲律宾"])  
  
print(markets)
```

选择合适的方法

增加一个元素使用 `append()`，增加多个元素使用 `extend()`；二者都会修改原列表，返回值是 `None`。思考 `markets.append(["新加坡", "菲律宾"])` 的结果。

insert()与加法：插入原列表还是生成新列表

在指定位置插入

`insert()` 修改原列表，原位置及其后的元素依次后移。

```
markets = [  
    "泰国", "马来西亚",  
    "新加坡"  
]  
  
markets.insert(1, "越南")  
  
print(markets)
```

使用加法组合列表

列表加法生成一个新列表，不修改参与运算的原列表。

```
markets = ["泰国", "越南"]  
  
new_markets = markets + [  
    "菲律宾", "新加坡"  
]  
  
print(markets)  
print(new_markets)
```

如何选择

需要直接更新原列表时使用 `insert()`；需要保留原列表并得到一个新列表时，可以使用列表加法。

remove()与pop(): 按值还是按位置

按元素的值删除

`remove()` 删除第一个与指定值相等的元素。

```
markets = [  
    "泰国", "越南",  
    "泰国", "新加坡"  
]
```

```
markets.remove("泰国")  
  
print(markets)
```

按元素的位置取出

`pop()` 删除指定位置的元素，并返回被删除的元素。

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]
```

```
removed_market = markets.pop(1)  
  
print(removed_market)  
print(markets)
```

选择合适的方法

知道元素的值时使用 `remove()`；知道元素的位置，并且还要继续使用被删除的数据时，使用 `pop()`。省略索引的 `pop()` 会取出最后一个元素。

del与clear(): 删除列表元素还是清空列表

删除位置或区间

`del` 可以删除指定索引或切片对应的元素。

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]
```

```
del markets[1]  
print(markets)
```

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]
```

```
del markets[1:3]  
print(markets)
```

清空列表

`clear()` 删除全部元素, 但列表变量仍然存在。

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]
```

```
markets.clear()  
print(markets)
```

第3节小结

对列表元素进行修改、增加、删除有哪些实现范式。运行代码前, 先预测原列表是否改变、改变后是什么, 以及操作是否返回结果;

成员判断：某个市场是否在清单中

业务问题

拓展新的销售区域之前，需要先判断目标市场是否已经出现在清单中。

```
markets = [  
    "泰国", "越南",  
    "马来西亚", "新加坡"  
]  
  
print("越南" in markets)      # True  
print("日本" in markets)      # False  
print("日本" not in markets)  # True
```

表达式的结果

`in` 判断元素是否存在，`not in` 判断元素是否不存在。表达式返回 `True` 或 `False`，不会修改原列表。

查询数量、次数与位置

数量与出现次数

`len` 统计全部元素的数量，`count()` 统计指定元素出现的次数。

```
markets = [
    "泰国", "越南",
    "泰国", "新加坡"
]

print(len(markets))          #
↪ 4
print(markets.count("泰国")) #
↪ 2
```

第一次出现的位置

`index()` 返回指定元素第一次出现的索引。

```
markets = [
    "泰国", "越南",
    "泰国", "新加坡"
]

print(markets.index("泰国"))
↪ # 0
print(markets.index("新加坡"))
↪ # 3
```

使用`index()`前先判断

如果元素不存在，`index()` 会产生错误。数据是否存在还不确定时，可以先使用 `in` 进行判断。

数值列表的基本统计

```
daily_orders = [
    128, 156, 143, 173, 151
]

total_orders = sum(daily_orders)
average_orders = (
    total_orders / len(daily_orders)
)
lowest_orders = min(daily_orders)
highest_orders = max(daily_orders)

print(total_orders)           # 751
print(average_orders)         # 150.2
print(lowest_orders)          # 128
print(highest_orders)         # 173
```

业务含义

- `len()` : 记录数量;
- `sum()` : 订单总量;
- `min()` : 最低订单量;
- `max()` : 最高订单量。

平均值

Python没有专门针对列表的平均值函数, 可以使用 `sum(data)/len(data)` 。

sort(): 直接调整原列表的顺序

```
daily_sales = [
    12600, 9800, 15800, 11200
]

daily_sales.sort()
print(daily_sales)
# [9800, 11200, 12600, 15800]

result = daily_sales.sort(reverse=True)
print(daily_sales)
# [15800, 12600, 11200, 9800]

print(result)    # None
```

升序与降序

sort() 默认按升序排列。设置 reverse=True 后，列表按降序排列。不要编写 `daily_sales = daily_sales.sort()`。sort() 直接修改原列表，返回值是 None，这样会使变量失去原来的列表。

sort()与sorted(): 是否保留原列表

```
daily_sales = [12600, 9800, 15800, 11200]

ranked_sales = sorted(daily_sales, reverse=True)

print(daily_sales)
# [12600, 9800, 15800, 11200]

print(ranked_sales)
# [15800, 12600, 11200, 9800]
```

两种排序方式的区别

操作	原列表	返回结果
<code>list.sort()</code>	直接改变	<code>None</code>
<code>sorted(list)</code>	保持不变	排序后的新列表

需要直接调整原列表时使用 `sort`。需要保留原始数据并生成排序结果时用 `sorted`。

key参数：指定排序依据

业务问题

页面展示空间有限，需要按照活动名称的长度排列营销活动。

```
campaigns = [  
    "双十一预热",  
    "新品",  
    "国庆促销",  
    "会员日"  
]  
  
by_length = sorted(campaigns, key=len)  
  
print(by_length)  
# ['新品', '会员日',  
#  '国庆促销', '双十一预热']
```

如何理解key

默认情况下，Python直接比较列表中的元素。

设置 `key=len` 后，Python比较每个字符串的长度。

降序排列

还可以同时设置 `key=len` 和 `reverse=True`，让较长的名称排在前面。

这里使用的是 `sorted()`，所以 `campaigns` 保持原来的顺序。

课堂练习：生成业务数据摘要

```
markets = ["泰国", "越南", "马来西亚",
↪ "新加坡"]
```

```
daily_orders = [128, 156, 143, 173,
↪ 151]
```

```
# 是否包含“越南”
```

```
has_vietnam = _____
```

```
# 目标市场数量
```

```
market_count = _____
```

```
# 订单总量与最高订单量
```

```
total_orders = _____
```

```
highest_orders = _____
```

```
# 保留原列表，生成降序结果
```

```
ranked_orders = _____
```

可使用的工具

- in
- len()
- sum()
- max()
- sorted()

完成后检查

- 每个表达式返回什么类型？
- `daily_orders` 是否改变？
- 如果改用 `sort()`，结果有什么不同？

查询和统计通常返回一个新结果。排序时需要特别判断是直接修改原列表，还是生成一个新的有序列表。

一个列表，两个变量名

业务问题

公司希望保留原来的目标市场清单，同时建立一份调整后的新清单。

```
original_markets = ["泰国", "越南", "马来西亚", "新加坡"]

new_markets = original_markets
new_markets.append("菲律宾")

print(original_markets)
print(new_markets)
```

运行结果

两个变量输出的内容相同，`original_markets`中也增加了“菲律宾”。语句 `new_markets = original_markets` 是引用赋值，没有创建新列表，两个变量名指向同一个列表，通过其中任何一个变量修改列表，另一个变量看到的内容也会改变。

copy(): 建立独立的列表版本

```
original_markets = ["泰国", "越南", "马来西亚", "新加坡"]

new_markets = original_markets.copy()
new_markets.append("菲律宾")

print(original_markets)
# ['泰国', '越南', '马来西亚', '新加坡']

print(new_markets)
# ['泰国', '越南', '马来西亚', '新加坡', '菲律宾']
```

观察结果

copy() 创建一个新列表。修改 new_markets 不会改变 original_markets。需要保留原始数据，同时尝试修改、排序或删除数据时，可先用 copy() 建立一个独立版本。

建立列表副本的三种写法

```
original_markets = ["泰国", "越南", "马来西亚", "新加坡"]  
  
copy_1 = original_markets.copy()  
copy_2 = original_markets[:]  
copy_3 = list(original_markets)
```

三种写法的含义

写法	说明
<code>original.copy()</code>	直接表达“复制列表”的意图，推荐初学者使用。
<code>original[:]</code>	通过完整切片生成一个新列表。
<code>list(original)</code>	使用 <code>list()</code> 根据原列表创建新列表。

这三种写法对于由字符串、整数或浮点数组成的**一维列表**效果相同。

浅复制的边界：列表中还有列表

```

original_records = [
    ["泰国", 128],
    ["越南", 156]
]

backup_records = (
    original_records.copy()
)

backup_records[0][1] = 200

print(original_records)
# [['泰国', 200],
#   ['越南', 156]]

import copy

backup_records =
    ↪ copy.deepcopy(original_records)

```

为什么原数据也改变了

`copy()` 创建了新的外层列表, 但其中的两个内部列表仍然与原列表共享。这种复制方式称为浅复制。

完全独立的复制

嵌套数据需要完全独立的副本时, 可以使用标准库中的 `copy.deepcopy()`。

赋值、浅复制、深复制

普通赋值会共享同一个列表。
`copy()` 可以独立复制一维列表。列表中还包含列表时, 使用 `copy.deepcopy()`。

业务实例：建立新版目标市场清单

```
original_markets = ["泰国", "越南", "马来西亚", "新加坡"]

# 根据原方案建立新版本
revised_markets = original_markets.copy()

# 调整新版本
revised_markets.remove("新加坡")
revised_markets.append("菲律宾")
revised_markets.insert(0, "印度尼西亚")

print("原方案: ", original_markets)
print("新方案: ", revised_markets)
```

运行结果

原方案仍然包含“新加坡”。新方案删除“新加坡”，并增加“菲律宾”和“印度尼西亚”。保留原始版本可以核对修改前后的差异，也能避免试验性修改破坏原始数据。

嵌套列表：表示业务表格

市场业务数据

市场	订单量	增长率
泰国	1280	0.08
越南	1560	0.12
马来西亚	1100	0.05

```
market_data = [  
    ["泰国", 1280, 0.08],  
    ["越南", 1560, 0.12],  
    ["马来西亚", 1100, 0.05]  
]
```

如何理解这个结构

外层列表表示整张业务表。每个内部列表表示一条市场记录，内部元素依次表示市场名称、订单量和增长率。

数据格式需要保持一致

每条记录的字段顺序应当相同。索引 1 始终保存订单量，索引 2 始终保存增长率。

双重索引：先定位记录，再定位字段

```
market_data = [  
    ["泰国", 1280, 0.08],  
    ["越南", 1560, 0.12],  
    ["马来西亚", 1100, 0.05]  
]  
  
first_record = market_data[0]  
market = market_data[0][0]  
orders = market_data[0][1]  
growth = market_data[0][2]  
  
print(first_record)  
print(market)  
print(orders)  
print(growth)
```

第一个索引

`market_data[0]` 取得第1条记录：`["泰国", 1280, 0.08]`。

第二个索引

在第1条记录中：

- `[0]` 表示市场名称，
- `[1]` 表示订单量，
- `[2]` 表示增长率。

理解代码

`market_data[0][1]` 读作：第1条市场记录中的订单量。两个层级的索引都从 0 开始。

修改字段与增删记录

```
market_data = [  
    ["泰国", 1280, 0.08],  
    ["越南", 1560, 0.12],  
    ["马来西亚", 1100, 0.05]  
]  
  
# 修改越南市场的订单量和增长率  
market_data[1][1] = 1680  
market_data[1][2] = 0.14  
  
# 增加一条市场记录  
market_data.append(["菲律宾", 920, 0.15])  
  
# 删除第1条市场记录  
removed_record = market_data.pop(0)  
  
print(removed_record)  
print(market_data)
```

双重索引修改一条记录中的某个字段。`append()` 和 `pop()` 作用于外层列表，因此增加或删除的是整条记录。

提取一列数据并完成统计

```

market_data = [
    ["泰国", 1280, 0.08],
    ["越南", 1560, 0.12],
    ["马来西亚", 1100, 0.05]
]

order_counts = [
    market_data[0][1],
    market_data[1][1],
    market_data[2][1]
]

total_orders = sum(order_counts)
average_orders = (
    total_orders / len(order_counts)
)

print(total_orders)          # 3940
print(average_orders)        # 1313.33...
print(max(order_counts))    # 1560

```

计算步骤

先从每条市场记录中取得订单量，再把这些数值组成一个一维列表。已有的 `sum()`、`len()` 和 `max()` 便可以继续使用。

当前写法的局限

市场数量改变后，提取订单量的代码也要手动修改。

后续学习循环后，可以让程序自动处理任意数量的记录。

综合实践：调整市场业务数据

```
market_data = [  
    ["泰国", 1280, 0.08],  
    ["越南", 1560, 0.12],  
    ["马来西亚", 1100, 0.05]  
]
```

任务要求

在不修改 `market_data` 的情况下，建立一份调整后的市场数据。

- 1 建立完全独立的数据副本；
- 2 把越南订单量改为 1680，增长率改为 0.14；
- 3 增加菲律宾记录：["菲律宾", 920, 0.15]；
- 4 删除马来西亚记录；
- 5 计算调整后的订单总量和最高订单量。

提示

动手前先分解：先确定每项业务要求对应哪个列表操作，再确定操作的行索引和字段索引，最后编写并运行代码。

可使用的工具：`copy.deepcopy()`、
双重索引、`append()`、`pop()`、
`sum()` 和 `max()`。

综合实践：参考实现与结果核对

```
import copy

revised_data = copy.deepcopy(market_data)

revised_data[1][1] = 1680
revised_data[1][2] = 0.14

revised_data.append(["菲律宾", 920, 0.15])

removed_record = revised_data.pop(2)

order_counts = [
    revised_data[0][1],
    revised_data[1][1],
    revised_data[2][1]
]

print(sum(order_counts)) # 3880
print(max(order_counts)) # 1680
```

结果核对

- 原始数据保持不变；
- 新数据包含泰国、越南和菲律宾；
- 订单总量为 3880 ；
- 最高订单量为 1680 ；
- 删除的记录是马来西亚。

固定索引只适合少量且数量确定的数据。学习循环后，程序可以逐条处理列表中的全部记录。

本讲知识点梳理与总结

任务	要解决的问题	主要工具
保存数据	用一个变量保存多个同类数据	方括号 <code>[]</code> 、 <code>list()</code>
定位数据	取得一个元素或连续多个元素	索引、切片
更新数据	修改、增加或删除元素	索引赋值、列表方法、 <code>del</code>
查询与统计	判断是否存在并计算数量或汇总值	<code>in</code> 、 <code>len()</code> 、 <code>count()</code> 、 <code>sum()</code>
调整顺序	直接排序或生成新的排序结果	<code>sort()</code> 、 <code>sorted()</code>
保留原数据	建立可以独立修改的新版本	<code>copy()</code> 、 <code>copy.deepcopy()</code>
表示表格数据	在一条记录中保存多个字段	嵌套列表、双重索引

编写列表代码时的检查顺序

先确定数据应该怎样组织，再确定如何定位所需元素。随后判断原列表是否改变、代码返回什么结果，最后运行代码核对业务结果。

列表操作的结果

操作类型	典型操作	原列表	返回的结果
直接修改	索引赋值、 <code>append()</code> 、 <code>extend()</code> 、 <code>remove()</code> 、改变 <code>del</code> 、 <code>clear()</code>		<code>None</code>
删除并取出	<code>pop()</code>	改变	被删除的元素
原地排序	<code>sort()</code>	改变	<code>None</code>
生成新列表	切片、 <code>+</code> 、 <code>copy()</code> 、 <code>copy.deepcopy()</code> 、不变 <code>sorted()</code>		一个新列表
查询或统计	<code>in</code> 、 <code>len()</code> 、 <code>count()</code> 、 <code>sum()</code> 、 <code>max()</code>	不变	布尔值或数值

课后实践：整理促销周销售数据

原始数据

列表依次保存促销活动前5天的销售额，单位为元。核对业务记录后发现，第2天的销售额应为 10800 元。第6天的销售额为 14200 元。

```
raw_sales = [  
    12600,  
    9800,  
    15800,  
    11200,  
    13400  
]
```

完成要求

本题只使用本讲已经学习的列表操作。完成后，`raw_sales` 应当保持不变。

实践任务

- 1 建立 `raw_sales` 的独立副本；
- 2 在副本中更正第2天的数据；
- 3 把第6天的销售额添加到列表末尾；
- 4 计算总销售额、平均销售额和最高销售额；
- 5 生成一个按销售额降序排列的新列表；
- 6 输出原始数据、调整后数据和排序结果。

课后实践的验收标准

三个列表的预期结果

原始数据、调整后数据和排序结果应当相互独立。

```
raw_sales = [  
    12600, 9800, 15800,  
    11200, 13400  
]
```

```
working_sales = [  
    12600, 10800, 15800,  
    11200, 13400, 14200  
]
```

```
ranked_sales = [  
    15800, 14200, 13400,  
    12600, 11200, 10800  
]
```

统计结果

指标	结果
总销售额	78000
平均销售额	13000
最高销售额	15800

自我检查

修改的是哪一个列表？

`sorted()` 是否改变了调整后的列表？

每个变量保存的是什么类型的数据？

列表推导式：根据已有列表生成新列表

业务任务

商品参加九折促销，需要根据原价列表生成折后价列表，同时保留原价。

```
prices = [89.0, 129.0, 59.0, 199.0]

discounted_prices = [round(price * 0.9, 2) for price in prices]

print(prices)
# [89.0, 129.0, 59.0, 199.0]

print(discounted_prices)
# [80.1, 116.1, 53.1, 179.1]
```

基本结构

[表达式 for 元素 in 可迭代对象]

对原列表中的每个元素计算一次表达式，并把结果依次放入新列表。原列表保持不变。列表推导式建立在循环基础上。当前先理解它的作用，学习 for 循环后再要求熟练编写。

列表推导式：筛选符合条件的数据

业务任务

从每日销售额中筛选出不低于 13000 元的数据。

```
daily_sales = [  
    12600, 10800, 15800,  
    11200, 13400, 14200  
]  
  
high_sales = [sale for sale in daily_sales if sale >= 13000]  
  
print(high_sales)  
# [15800, 13400, 14200]
```

带条件的基本结构

[元素 for 元素 in 列表 if 条件]

程序依次检查原列表中的元素，只把满足条件的元素放入新列表。简单转换和筛选适合使用列表推导式。处理步骤较多或条件较复杂时，普通循环通常更容易阅读和检查。

列表乘法与共享引用

重复不可变元素

列表乘法可以快速重复已有元素。

```
status = ["待处理"] * 4

print(status)
# ['待处理', '待处理',
#  '待处理', '待处理']
```

适用情况

重复字符串、整数等不可变数据时，这种写法简单直观。

嵌套列表中的问题

列表乘法会重复引用同一个内部列表。

```
grid = [[0] * 3] * 2

grid[0][0] = 1

print(grid)
# [[1, 0, 0],
#  [1, 0, 0]]
```

为什么两行都改变了

两行数据指向同一个内部列表。修改其中一行时，另一行看到的内容也会改变。

建立相互独立的内部列表

THE END